

Quick practice 1:

```
x <- 'thisIsGoodPractice'
```

Use ****stringr**** functions to transform x into the
following strings:

```
# 'thisIsGood'
```

```
str_sub(x, 1, 10)
```

```
index <- str_locate(x, 'thisIsGood')  
str_sub(x, index[1], index[2])
```

```
# 'practice'
```

```
str_to_lower(str_sub(x, -8, -1))
```

```
x_lower <- str_to_lower(x)  
index <- str_locate(x_lower, 'practice')  
str_sub(x_lower, index)
```

```
# 'GOOD'
```

```
str_sub(str_to_upper(x), 7, 10)  
str_to_upper(str_sub(x, 7, 10))
```

```
# 'thisthisthis'
```

```
str_dup(str_sub(x, 1, 4), 3)
```

```
# 'GOODGOODGOOD'
```

```
x_upper <- str_to_upper(x)  
str_sub(x_upper, 1, 10)
```

****Hint****: You'll need these:

```
# str_to_lower()
```

```
# str_to_upper()
```

```
# str_locate()
```

```
# str_sub()
```

```
# str_dup()
```

```
# -----
```

Quick practice 2:

Create the following objects:

```
x <- 'this_is_good_practice'
```

```
y <- c('hello', 'world')

# Use stringr functions to transform x and y into the following:

# "hello world"

hw <- paste(y, collapse = " ")

# "***hello world***"

width <- str_length(hw) + 6
str_pad(hw, pad = "*", side = "both", width = width)

str_pad(hw, width, "both", "*")

# c("this", "is", "good", "practice")

x_vector <- str_split(x, "_")[[1]]
x_vector

# "this is good practice"

x_sentence <- paste(x_vector, collapse = " ")

# "hello world, this is good practice"

paste(hw, x_sentence, sep = ", ")

# **Hint**: You'll need these:
# str_trim()
# str_pad()
# paste()
# str_split()
```

Your turn 1:

1) reverseString(s): Write a function that returns the
string s in reverse order.

```
reverseString <- function(s) {  
  result <- ""  
  for (i in 1:str_length(s)) {  
    letter <- str_sub(s, -i, -i)  
    result <- paste(result, letter, sep = "")  
  }  
  return(result)  
}
```

```
reverseString <- function(s) {  
  result <- c()  
  for (i in 1:str_length(s)) {  
    letter <- str_sub(s, -i, -i)  
    result <- c(result, letter)  
  }  
  return(paste(result, collapse = ""))  
}
```

```
reverseString <- function(s) {  
  letters <- str_split(s, "")[[1]]  
  return(paste(rev(letters), collapse = ""))  
}
```

2) isPalindrome(s): Write a function that returns TRUE if
the string s is a Palindrome and FALSE otherwise.

```
isPalindrome <- function(s) {  
  return(s == reverseString(s))  
}
```

```
# -----
```

```
# Quick practice 3:
```

```
# Use stringr functions to answer the following questions
```

```
# about the fruit vector:
```

```
# 1. How many fruit have the string "rr" in it?
```

```
sum(str_detect(fruit, "rr"))
```

```
# 2. Which fruit end with string "fruit"?
```

```
fruit[str_detect(fruit, "fruit$")]
```

```
# 3. Which fruit contain more than one "o" character?
```

```
fruit[str_count(fruit, "o") > 1]
```

```
# -----
```

```
# Your turn 2:
```

```
sensorText <- function(text, words) {
```

```
  # Make lower case everything
```

```
  result <- str_to_lower(text)
```

```
  words <- str_to_lower(words)
```

```
  # For each word, replace with asterisks with same length as the word
```

```
  for (word in words) {
```

```
    replacement <- str_dup("?", str_length(word))
```

```
    result <- str_replace_all(result, word, replacement)
```

```
  }
```

```
  # Go back and restore any non-censored characters from the original text
```

```
  result <- str_split(result, " ")[[1]]
```

```
  text <- str_split(text, " ")[[1]]
```

```
  for (i in seq(length(result))) {
```

```
    if (result[i] != '?') {
```

```
      result[i] <- text[i]
```

```
    }
```

```
  }
```

```
}
```

```
  return(paste(result, collapse = " "))
```

```
}
```

1) sortString(s): Write the function sortString(s) that takes
a string s and returns back an alphabetically sorted string.

```
test_sortString <- function() {  
  cat("Testing sortString()...")  
  stopifnot(sortString("cba") == "abc")  
  stopifnot(sortString("abedhg") == "abdegh")  
  stopifnot(sortString("AbacBc") == "aAbBcc")  
  cat("Passed!\n")  
}
```

```
sortString <- function(s) {  
  letters <- str_split(s, "")[[1]]  
  letters <- str_sort(letters)  
  return(paste(letters, collapse = ""))  
}
```

test_sortString()

2) areAnagrams(s1, s2): Write the function areAnagrams(s1, s2) that
takes two strings, s1 and s2, and returns TRUE if the strings are
anagrams and FALSE otherwise. **Treat lower and upper case as the
same letters**.

```
test_areAnagrams <- function() {  
  cat("Testing areAnagrams()...")  
  stopifnot(areAnagrams("", "") == TRUE)  
  stopifnot(areAnagrams("aabbccdd", "bbccdde") == FALSE)  
  stopifnot(areAnagrams("TomMarvoloRiddle", "IAmLordVoldemort") == TRUE)  
  cat("Passed!\n")  
}
```

```
areAnagrams <- function(s1, s2) {  
  s1 <- str_to_lower(s1)  
  s2 <- str_to_lower(s2)  
  return(sortString(s1) == sortString(s2))  
}
```

test_areAnagrams()